

CPP - Module 00

De cero a entenderlo todo de verdad

ÍNDICE

PARTE 1 — Antes de escribir una sola línea

1. Qué es C++ y en qué se diferencia de C
2. El primer cambio visible — adiós printf, hola cout
3. Qué es un namespace y por qué existe std::

PARTE 2 — Strings: el reemplazo de char* 4. El problema con las cadenas en C 5. std::string — la solución 6. Todo lo que puedes hacer con un string 7. Cómo recorrer un string — índices e iteradores 8. Convertir un string a mayúsculas

PARTE 3 — Clases: agrupar datos y funciones 9. El problema que resuelven las clases 10. Qué es una clase — el molde 11. La diferencia entre clase y objeto 12. Private y public — quién puede tocar qué 13. Cómo se separa el código — .hpp y .cpp 14. Include guards y pragma once

PARTE 4 — Los constructores y destructores 15. El problema sin constructor 16. El constructor — la función que nace sola 17. El destructor — la función que muere sola 18. El puntero this — quién soy yo

PARTE 5 — Getters, setters y encapsulación 19. Por qué no dejar los atributos públicos 20. El getter — la ventanilla de lectura 21. El setter — la ventanilla de escritura

PARTE 6 — Static y const en clases 22. El atributo static — compartido por todos 23. El método const — promesa de no tocar nada 24. El atributo static const — la constante de clase

PARTE 7 — Entrada y salida 25. std::cin — leer del teclado 26. El problema de mezclar cin y getline 27. iomanip — formatear columnas

PARTE 8 — argc y argv — argumentos del programa 28. Cómo recibe el programa datos desde la terminal

PARTE 9 — Los ejercicios explicados uno a uno 29. Ex00: Megaphone 30. Ex01: PhoneBook — Contact 31. Ex01: PhoneBook — la clase PhoneBook 32. Ex02: Account — static en acción

Glosario completo

PARTE 1 — ANTES DE ESCRIBIR UNA SOLA LÍNEA

1. Qué es C++ y en qué se diferencia de C

Ya sabes C. Ya sabes que puedes reservar memoria, escribir funciones, trabajar con punteros. C++ no tira nada de eso — lo conserva todo y añade una capa encima.

Esa capa se llama **Programación Orientada a Objetos** (OOP). La idea central es simple: en lugar de tener datos por un lado y funciones que los manipulan por otro, C++ te permite empaquetar ambas cosas juntas en una **clase**.

En C hacías esto:

```
// Datos separados de las funciones
struct Contacto {
    char nombre[50];
    char telefono[20];
};

void mostrar_contacto(struct Contacto c) { ... }
void guardar_contacto(struct Contacto *c, char *nombre) { ... }
```

En C++ puedes hacer esto:

```
// Datos y funciones juntos
class Contacto {
    std::string _nombre;
    std::string _telefono;

    void mostrar() { ... }      // la función PERTENECE al objeto
    void guardar(std::string n) { ... }
};
```

El objeto `Contacto` lleva sus propias funciones consigo. Cuando tienes un `Contacto`, ya

sabes lo que puede hacer.

Además, en 42 usas el estándar **C++98** — la versión más antigua. Esto es intencionado: te obliga a entender los fundamentos sin atajos modernos.

2. El primer cambio visible — adiós printf, hola cout

Lo primero que choca al pasar de C a C++ es que `printf` está prohibido. En su lugar usas `std::cout`.

```
// C
printf("Hola %s, tienes %d años\n", nombre, edad);

// C++
std::cout << "Hola " << nombre << ", tienes " << edad << " años" << std::endl;
```

`std::cout` es un **objeto** que representa la pantalla. El operador `<<` manda datos hacia él. Puedes encadenar tantos `<<` como quieras — cada uno manda su dato al stream y devuelve el mismo stream para que puedas seguir encadenando.

Visualizando la ejecución:

```
std::cout << "Hola " << nombre << std::endl;
```

Paso 1: "Hola " va al stream → pantalla muestra: Hola

Paso 2: nombre ("Alice") va al stream → pantalla muestra: Hola Alice

Paso 3: `std::endl` va al stream → pantalla muestra: Hola Alice\n

`std::endl` hace dos cosas: salta de línea y vacía el buffer (el buffer es una zona temporal donde se acumula lo que vas a imprimir antes de que aparezca en pantalla). En la práctica, para 42, usa siempre `std::endl`.

Para tener acceso a `std::cout` necesitas incluir:

```
#include <iostream>
```

3. Qué es un namespace y por qué existe std::

Imagina que en tu empresa hay dos empleados llamados García. Cuando alguien dice "llama a García", nadie sabe a cuál. Para distinguirlos dices "García de ventas" o "García de

contabilidad".

Un **namespace** es exactamente eso — una etiqueta que agrupa nombres para evitar conflictos.

La librería estándar de C++ mete todo dentro de un namespace llamado `std`. Por eso escribes `std::cout`, `std::string`, `std::endl` — estás diciendo "el `cout` que pertenece a `std`".

El `::` se llama **operador de resolución de ámbito**. Significa "lo que está dentro de".
`std::cout` se lee: "el `cout` que está dentro del namespace `std`".

Existe un atajo para no escribir `std::` todo el rato:

```
using namespace std;
cout << "Hola" << endl;    // ya no necesitas std::
```

Pero en 42 esto está prohibido en los headers y muy desaconsejado en general. La razón: si tu proyecto tiene dos namespaces con funciones del mismo nombre, el compilador no sabe cuál usar. Es una fuente de bugs difíciles de detectar.

La regla de oro: **escribe siempre `std::` explícitamente**.

PARTE 2 — STRINGS: EL REEMPLAZO DE CHAR*

4. El problema con las cadenas en C

En C las cadenas de texto eran arrays de `char` terminados en `'\0'`. El problema era que tenías que gestionar el tamaño tú mismo:

```
char nombre[20];           // tienes que adivinar el tamaño máximo
strcpy(nombre, "Alice");    // copia manual
strcat(nombre, " Smith");   // concatenación manual — peligrosa si se sale del array
int len = strlen(nombre);   // longitud manual
```

Si el string crecía más de 19 caracteres, desbordabas el array y corrompías memoria. Si lo hacías demasiado grande, desperdiciabas espacio.

5. std::string — la solución

En C++, `std::string` es una clase que gestiona todo esto automáticamente. Tú solo escribes texto:

```
std::string nombre = "Alice";
nombre += " Smith";           // concatenación automática, sin desbordamientos
int len = nombre.length();    // longitud automática
```

Por dentro, `std::string` reserva y libera memoria según lo necesite. Tú no te preocupas del tamaño ni del `'\0'`. Para usarla necesitas:

```
#include <string>
```

Grábate esto: en C++ usas `std::string`, no `char *`. Si ves `char *` para texto en C++, algo está mal.

6. Todo lo que puedes hacer con un string

Vamos con ejemplos concretos usando siempre el mismo string:

```
std::string frase = "Hola Alice mundo";
```

Tamaño:

```
frase.length()    // 16
frase.size()       // también 16 — son equivalentes
```

Comprobar si está vacío:

```
std::string vacio = "";
vacio.empty()      // true
frase.empty()      // false
```

Acceder a un carácter por posición:

```
frase[0]           // 'H'
frase[5]           // 'A'
frase[15]          // '\0'
// los índices empiezan en 0, igual que en C
```

Comparar:

```
std::string a = "Hola";
std::string b = "Hola";
a == b      // true – en C necesitabas strcmp, aquí basta con ==
a != b      // false
```

Buscar una subcadena:

```
frase.find("Alice")    // devuelve 5 – la posición donde empieza "Alice"
frase.find("zombie")   // devuelve std::string::npos – no encontrado
```

`std::string::npos` es una constante especial que significa "no encontrado". Siempre comprueba contra ella cuando uses `find`:

```
if (frase.find("Alice") == std::string::npos)
    std::cout << "No encontrado" << std::endl;
```

Extraer parte del string:

```
frase.substr(5, 5)     // desde posición 5, coge 5 caracteres → "Alice"
frase.substr(11)       // desde posición 11 hasta el final → "mundo"
```

Concatenar:

```
std::string a = "Hola";
std::string b = " mundo";
std::string c = a + b;    // "Hola mundo"
a += " Alice";           // a ahora es "Hola Alice"
```

Borrar parte del string:

```
frase.erase(5, 6)       // borra 6 caracteres desde posición 5 → "Hola mundo"
```

Insertar texto en una posición:

```
frase.insert(5, "Bob ")  // inserta "Bob " en posición 5 → "Hola Bob Alice mundo"
```

7. Cómo recorrer un string — índices e iteradores

Hay dos formas de recorrer un string carácter por carácter. Ambas producen el mismo resultado.

Forma 1 — con índice (la más simple, igual que C):

```
std::string frase = "Hola Alice";

for (int i = 0; i < (int)frase.length(); i++) {
    std::cout << frase[i] << std::endl;
}
// imprime: H o l a   A l i c e (uno por línea)
```

El cast `(int)` antes de `frase.length()` evita un warning — `length()` devuelve `size_t` (entero sin signo) y `i` es `int` (con signo). Mezclarlos da warning. El cast lo silencia.

Forma 2 — con iterador (la forma C++ estándar):

Un iterador es como un cursor que señala a un elemento dentro de una colección. Para moverlo hacia adelante usas `it++`. Para leer el elemento al que apunta usas `*it`.

```
std::string frase = "Hola Alice";
std::string::iterator it;

for (it = frase.begin(); it != frase.end(); it++) {
    std::cout << *it << std::endl;
}
```

Visualizando el iterador:

```
frase = "Hola Alice"
      ^         ^
      |         |
begin()   end()
(apunta a 'H') (apunta al lugar DESPUÉS de 'e')
```

`end()` no apunta al último carácter — apunta a la posición imaginaria después de él. Por eso el bucle es `it != end()` y no `it <= end()`.

Para strings simples, los índices son más claros. Los iteradores son la forma universal de C++ para recorrer cualquier tipo de colección y los usarás más en módulos avanzados.

8. Convertir un string a mayúsculas

`std::string` no tiene un método directo para esto. Tienes que recorrer carácter por carácter usando `toupper()` de `<cctype>`:

```
#include <cctype>

std::string frase = "Hola Alice";

for (int i = 0; i < (int)frase.length(); i++) {
    frase[i] = toupper(frase[i]);
}
// frase ahora es "HOLA ALICE"
```

`toupper(c)` toma un `char` y devuelve su versión mayúscula. Si ya es mayúscula, lo deja igual. Si es un número o símbolo, también lo deja igual. Solo afecta a las letras minúsculas.

`tolower(c)` hace lo mismo pero al revés — convierte a minúscula.

El `ex00` (Megaphone) usa exactamente esto — recibe los argumentos del programa y los imprime todos en mayúsculas.

PARTE 3 — CLASES: AGRUPAR DATOS Y FUNCIONES

9. El problema que resuelven las clases

Imagina que tienes que hacer la agenda telefónica del `ex01`. En C harías algo así:

```
// Los datos aquí
char nombre[50];
char apellido[50];
char telefono[20];

// Las funciones allá
void mostrar_contacto(char *nombre, char *apellido, char *telefono) { ... }
void pedir_datos(char *nombre, char *apellido, char *telefono) { ... }
```

El problema: los datos y las funciones están separados. Si tienes 8 contactos, tienes 24 variables sueltas por ahí. Si quieres pasar un contacto a una función, tienes que pasar las 3 variables. Si el contacto cambia (añades un campo), tienes que modificar todas las

funciones.

Las clases resuelven esto agrupándolo todo:

```
class Contact {
    std::string _nombre;
    std::string _apellido;
    std::string _telefono;

    void mostrar();          // la función conoce _nombre, _apellido y _telefono
    void pedirDatos();       // directamente — son suyos
};
```

Ahora `Contact` es una unidad completa. Lleva sus datos y sus funciones juntos.

10. Qué es una clase — el molde

Una clase es un **molde** para crear objetos. El molde define qué datos tiene cada objeto y qué puede hacer.

La analogía perfecta: una clase es el plano de un apartamento. El plano en sí no es un apartamento — es la descripción de cómo construirlo. Cada apartamento construido siguiendo ese plano es un **objeto** (también llamado instancia).

```
class Contact {                // ← el plano
    std::string _nombre;
};

Contact alice;                 // ← apartamento 1
Contact bob;                   // ← apartamento 2
```

`alice` y `bob` son dos objetos distintos con sus propios datos, pero contruidos con el mismo molde. Si cambias el nombre de `alice`, el de `bob` no cambia.

El punto y coma después de `}` :

```
class Contact {
    // ...
};    // ← obligatorio en C++
```

En las funciones normales no pones `;` después de `}`. En las clases sí es obligatorio. Si lo olvidas, error de compilación críptico.

11. La diferencia entre clase y objeto

- **Clase** = el plano = Contact
- **Objeto** = el apartamento construido = alice, bob

La clase solo existe en tu código fuente. Los objetos existen en memoria cuando el programa se ejecuta.

EN EL CÓDIGO:	EN MEMORIA (durante la ejecución):
<code>class Contact</code>	<code>alice._nombre = "Alice"</code>
<code>{</code>	<code>alice._apellido = "Smith"</code>
<code>_nombre;</code>	
<code>_apellido;</code>	<code>bob._nombre = "Bob"</code>
<code>};</code>	<code>bob._apellido = "Jones"</code>

Puedes crear tantos objetos como quieras a partir del mismo molde.

12. Private y public — quién puede tocar qué

Aquí está una de las ideas más importantes de C++: **encapsulación**.

Por defecto, todo lo que está en una clase es privado — nadie de fuera puede tocarlo. Para que algo sea accesible desde fuera, tienes que marcarlo explícitamente como `public`.

```
class Contact {
    private:                                // nadie de fuera puede tocar esto
        std::string _nombre;
        std::string _telefono;

    public:                                  // esto sí es accesible desde fuera
        void mostrar();
        void setNombre(std::string n);
};
```

Si alguien intenta acceder a `_nombre` directamente desde fuera:

```
Contact alice;
alice._nombre = "Alice";    // ERROR de compilación — _nombre es private
alice.setNombre("Alice");   // correcto — setNombre es public
```

¿Por qué proteger los datos?

Sin protección, cualquiera puede poner cualquier basura en los atributos:

```
alice._nombre = "";           // nombre vacío — estado inválido
alice._telefono = "abc123";   // teléfono inválido
```

Con un setter puedes validar antes de guardar:

```
void Contact::setNombre(std::string n) {
    if (n.empty())           // si está vacío, no lo guardo
        return;
    _nombre = n;             // solo guardo si es válido
}
```

La clase controla su propio estado. Eso es la encapsulación.

La convención del guión bajo:

En 4.2, los atributos privados llevan `_` delante: `_nombre`, `_telefono`. No es obligatorio por el lenguaje — es una convención para distinguirlos visualmente de los parámetros y variables locales.

13. Cómo se separa el código — .hpp y .cpp

En C++ los archivos de clase se dividen en dos:

El .hpp (header / cabecera): declara qué existe. Dice qué tiene la clase y qué puede hacer. No dice cómo funciona — solo la “firma” de cada función.

El .cpp (implementación): define cómo funciona cada cosa declarada en el .hpp.

```
Contact.hpp → "Contact tiene _nombre y puede mostrar()"
Contact.cpp → "así es como funciona mostrar()"
```

Ejemplo completo:

```
// Contact.hpp
#pragma once
#include <string>
#include <iostream>

class Contact {
```

```

private:
    std::string _nombre;
    std::string _apellido;

public:
    Contact();
    ~Contact();
    void setNombre(std::string nombre);
    std::string getNombre(void) const;
};

```

```

// Contact.cpp
#include "Contact.hpp"

Contact::Contact() {
    _nombre = "";
    _apellido = "";
}

Contact::~~Contact() { }

void Contact::setNombre(std::string nombre) {
    _nombre = nombre;
}

std::string Contact::getNombre(void) const {
    return _nombre;
}

```

El `Contact::` antes de cada función en el `.cpp` es el **operador de resolución de ámbito** — le dice al compilador “esta función pertenece a la clase `Contact`”. Sin él, el compilador pensaría que son funciones sueltas sin relación con ninguna clase.

14. Include guards y pragma once

Si el mismo `.hpp` se incluye desde varios archivos `.cpp`, el compilador lo procesaría varias veces — y al encontrar la misma clase declarada dos veces, daría error.

Los **include guards** evitan esto.

Forma clásica:

```

#ifndef CONTACT_HPP

```

```
#define CONTACT_HPP

// ... contenido del header

#endif
```

La primera vez que el compilador lee `Contact.hpp : CONTACT_HPP` no existe → entra, lo define, procesa el contenido. Si lo intenta leer una segunda vez: `CONTACT_HPP` ya existe → se salta todo.

Forma moderna (la que usarás en 42):

```
#pragma once

// ... contenido del header
```

Una sola línea que hace lo mismo. Funciona en todos los compiladores modernos.

Regla importante: nunca pongas la implementación de funciones en un `.hpp`. Solo declaraciones. Si defines el cuerpo de una función en el header, ese código se copia en todos los archivos que incluyan el header — causando el error “multiple definition”.

PARTE 4 — LOS CONSTRUCTORES Y DESTRUCTORES

15. El problema sin constructor

Imagina que creas un objeto y no inicializas sus atributos:

```
class Contact {
    private:
        std::string _nombre;
        int         _edad;
};

Contact alice;
// ¿qué valor tiene alice._edad?
// En C++ un int sin inicializar contiene basura — lo que hubiera en esa memoria
```

Sin constructor, el objeto nace en un estado desconocido. Cualquier operación sobre él es

potencialmente peligrosa.

16. El constructor — la función que nace sola

El constructor es una función especial que se ejecuta **automáticamente** cuando se crea un objeto. Su trabajo es dejar el objeto en un estado válido desde el primer instante.

```
class Contact {
    private:
        std::string _nombre;
        int         _edad;

    public:
        Contact();    // declaración del constructor
};

Contact::Contact() {    // implementación
    _nombre = "";
    _edad   = 0;
    std::cout << "Contact creado" << std::endl;
}
```

Cuando haces `Contact alice;`, el constructor se llama automáticamente. Tú no lo llamas manualmente — simplemente ocurre.

Características del constructor:

- Mismo nombre que la clase — siempre
- Sin tipo de retorno — ni siquiera `void`
- Se llama automáticamente al crear el objeto
- Puede sobrecargarse — puedes tener varios con distintos parámetros

La lista de inicialización:

Hay una forma más eficiente de inicializar atributos. En lugar de asignar dentro del cuerpo, usas `: atributo(valor)` entre los paréntesis y el cuerpo:

```
Contact::Contact() : _nombre(""), _edad(0) {
    std::cout << "Contact creado" << std::endl;
}
```

Esto construye los atributos directamente con el valor correcto, en lugar de construirlos vacíos y luego sobrescribirlos. Para tipos simples no importa mucho. Para objetos complejos es más eficiente. Y para las **referencias** (que verás en ex03 de CPP01) es **obligatorio** — una referencia debe inicializarse en la lista, no en el cuerpo.

Constructor con parámetros:

```
Contact::Contact(std::string nombre, int edad) : _nombre(nombre), _edad(edad) { }

// uso:
Contact alice("Alice", 25);
```

Constructor por defecto:

Es el constructor sin parámetros. Lo necesitas en casos como el PhoneBook, donde tienes un array de 8 `Contact` — el compilador construye cada uno sin parámetros al crear el array.

17. El destructor — la función que muere sola

El destructor es lo opuesto del constructor. Se ejecuta **automáticamente** cuando el objeto va a ser destruido.

```
class Contact {
public:
    Contact();
    ~Contact();    // la tilde ~ indica destructor
};

Contact::~~Contact() {
    std::cout << "Contact destruido" << std::endl;
}
```

¿Cuándo se llama?

Cuando el objeto sale del scope `{ }` donde vivía:

```
{
    Contact alice;    // constructor llamado
    // ... se usa alice ...
}                    // destructor llamado aquí automáticamente
```

¿Para qué sirve en la práctica?

Si tu clase reservó memoria dinámica con `new`, el destructor es donde la liberas con `delete`. En el PhoneBook del ex01 no hay memoria dinámica, así que el destructor queda vacío — pero debe existir y declararse.

El orden de destrucción:

Los objetos se destruyen en orden **inverso** al que fueron creados (LIFO):

```
Contact a;    // creado primero
Contact b;    // creado segundo
Contact c;    // creado tercero
// al salir del scope:
// → c destruido primero
// → b destruido segundo
// → a destruido tercero
```

18. El puntero `this` — quién soy yo

Dentro de cualquier método de clase existe un puntero especial llamado `this`. Apunta al objeto concreto sobre el que se está ejecutando el método en ese momento.

```
void Contact::setNombre(std::string nombre) {
    this->_nombre = nombre;
}
```

Si tienes tres contactos y llamas a `alice.setNombre("Alice")`, dentro de ese método `this` apunta a `alice`. Si llamas a `bob.setNombre("Bob")`, `this` apunta a `bob`. El mismo código — pero `this` cambia según quién lo ejecuta.

¿Cuándo necesitas escribir `this->` explícitamente?

Cuando el nombre del parámetro coincide con el del atributo:

```
void Contact::setNombre(std::string _nombre) {    // ojo: mismo nombre
    _nombre = _nombre;                          // MAL — se asigna a sí mismo
    this->_nombre = _nombre;                      // BIEN — distingue atributo de parámetro
}
```

Por eso en 42 los atributos llevan `_` — para evitar esta ambigüedad y no necesitar `this->` constantemente.

PARTE 5 — GETTERS, SETTERS Y ENCAPSULACIÓN

19. Por qué no dejar los atributos públicos

Ya lo vimos en la sección 12, pero vale la pena repetirlo con un ejemplo más completo.

Si haces `_nombre` público en el PhoneBook, cualquiera puede poner lo que quiera:

```
Contact alice;
alice._nombre = "";           // nombre vacío — no debería existir un contacto así
alice._telefono = "no sé";    // teléfono inválido
```

El subject dice explícitamente: “A saved contact can’t have empty fields.” Si `_nombre` fuera público, tendrías que comprobarlo en cada sitio donde alguien pudiera modificarlo. Con un setter, la comprobación está en un solo lugar.

20. El getter — la ventanilla de lectura

Un getter es un método público que devuelve el valor de un atributo privado. Es como una ventanilla — no puedes entrar a la caja fuerte, pero puedes pedir la información a través de ella.

```
// en Contact.hpp
std::string getNombre(void) const;

// en Contact.cpp
std::string Contact::getNombre(void) const {
    return _nombre;
}
```

El `const` al final de la firma es una **promesa**: “este método no modifica ningún atributo del objeto”. Los getters solo leen — no escriben — así que siempre deben marcarse como `const`. Esto permite llamarlos sobre objetos que son `const`:

```
const Contact alice;           // objeto constante — no se puede modificar
alice.getNombre();              // funciona — getNombre es const
alice.setNombre("Alice");       // ERROR — setNombre no es const
```

La convención de nombres:

- Getter: `get + NombreAtributo` → `getNombre`, `getApellido`, `getTelefono`
- Setter: `set + NombreAtributo` → `setNombre`, `setApellido`, `setTelefono`

No es obligatorio por el lenguaje, pero es una convención tan universal que todos la esperan.

21. El setter — la ventanilla de escritura

Un setter es un método público que modifica el valor de un atributo privado con posibilidad de validar:

```
// en Contact.hpp
void setNombre(std::string nombre);

// en Contact.cpp
void Contact::setNombre(std::string nombre) {
    if (nombre.empty())
        return;           // rechazo valores vacíos
    _nombre = nombre;      // solo guardo si es válido
}
```

Con esta estructura, es imposible que `_nombre` quede vacío — el setter lo impide. La clase controla su propio estado.

PARTE 6 — STATIC Y CONST EN CLASES

22. El atributo static — compartido por todos

Un atributo normal existe en cada objeto:

```
Contact alice;    // alice tiene su propio _nombre
Contact bob;      // bob tiene su propio _nombre
// son cosas distintas en memoria
```

Un atributo `static` existe **una sola vez** para toda la clase, compartido por todos los objetos:

```

class Account {
    private:
        static int _nbAccounts;    // existe una sola vez para todos los Account
        int _amount;              // cada Account tiene el suyo propio
};

int Account::_nbAccounts = 0;    // inicialización FUERA de la clase, en el .cpp

```

Visualizando en memoria:

```

Account a;    Account b;    Account c;

a._amount = 100        b._amount = 200        c._amount = 50

Account::_nbAccounts = 3    ← uno solo, compartido

```

El ex02 (Account) usa atributos estáticos para llevar la cuenta del número total de cuentas y el dinero total depositado — datos que pertenecen a “todas las cuentas”, no a ninguna en particular.

La inicialización fuera de la clase:

Los atributos estáticos deben inicializarse fuera de la clase en el `.cpp`. Si no, el compilador da error “undefined reference”:

```

// Account.cpp
int Account::_nbAccounts = 0;    // obligatorio

```

23. El método `const` — promesa de no tocar nada

Ya lo vimos en los getters, pero aquí la explicación completa.

Un método marcado como `const` al final de su firma promete no modificar ningún atributo del objeto:

```

bool Contact::isEmpty(void) const {
    return _nombre.empty();
    // _nombre = "algo";    ← si añadieras esto, ERROR de compilación
}

```

Si intentas modificar un atributo dentro de un método `const`, el compilador da error

inmediatamente. Es una red de seguridad.

Regla práctica: todo getter y todo método que solo lee datos → márcalo como `const`. Los setters no pueden ser `const` porque modifican atributos.

24. El atributo `static const` — la constante de clase

Combinando ambos modificadores se obtiene una constante que pertenece a la clase:

```
class PhoneBook {
    private:
        static const int MAX_CONTACTS = 8;
};
```

- `static` — existe una sola vez, no por objeto
- `const` — nunca cambia

Para tipos enteros (`int`, `short` ...) C++ permite inicializarlo directamente en el header. Para otros tipos debes inicializarlo fuera.

Esto es más limpio que un `#define` de C porque está dentro del ámbito de la clase — no contamina el espacio de nombres global.

PARTE 7 — ENTRADA Y SALIDA

25. `std::cin` — leer del teclado

`std::cin` es el objeto que representa la entrada del teclado. Funciona igual que `cout` pero al revés — con `>>`:

```
std::string nombre;
std::cin >> nombre;    // espera a que el usuario escriba y pulse Enter
```

Problema: `std::cin >>` lee hasta el primer espacio. Si el usuario escribe "Hola mundo", solo guardará "Hola".

Para leer una línea entera incluyendo espacios, usas `std::getline`:

```
std::string frase;
std::getline(std::cin, frase); // lee toda la línea hasta el Enter
```

26. El problema de mezclar cin y getline

Este es el bug más clásico en los ejercicios de C++. Cuando mezclas `cin >>` con `std::getline`, obtienes resultados raros:

```
int numero;
std::cin >> numero;           // el usuario escribe "5" y pulsa Enter
                               // cin lee el "5" pero deja el '\n' en el buffer

std::string nombre;
std::getline(std::cin, nombre); // lee el '\n' que quedó → nombre queda vacío!
```

Trazando lo que pasa:

```
Buffer del teclado antes de cin>>:  [5][\n]
                                     ↑
                               cin >> lee esto

Buffer después de cin>>:             [\n]
                                     ↑
                               getline lee esto (el \n que quedó)

nombre = "" ← ¡vacío!
```

La solución: limpiar el buffer después de `cin >>`:

```
std::cin >> numero;
std::cin.ignore();           // ignora el '\n' que quedó
std::getline(std::cin, nombre); // ahora sí lee correctamente
```

En el PhoneBook, para evitar este problema, usa `std::getline` para todo y no mezcles con `cin >>`.

27. iomanip — formatear columnas

El ex01 (PhoneBook) pide mostrar los contactos en columnas de exactamente 10 caracteres, alineados a la derecha, separados por `|`. Para esto necesitas:

```
#include <iomanip>
```

std::setw(n) — establece el ancho del siguiente campo:

```
std::cout << std::setw(10) << "Alice" << std::endl;
// imprime: "      Alice" (5 espacios + "Alice" = 10 caracteres total)
```

Si el texto tiene menos de 10 caracteres, rellena con espacios por la izquierda. Si tiene más de 10, **no lo trunca** — eso tienes que hacerlo tú.

std::right y **std::left** — alineación:

```
std::cout << std::right << std::setw(10) << "Alice"; // "      Alice"
std::cout << std::left  << std::setw(10) << "Alice"; // "Alice      "
```

std::right es el comportamiento por defecto. A diferencia de **setw**, **std::right** y **std::left** **persisten** — una vez que los activas, se aplican a todos los campos siguientes.

setw solo afecta al siguiente campo:

```
std::cout << std::setw(10) << "Alice" << "Bob";
// imprime: "      AliceBob"
// setw solo afectó a "Alice", no a "Bob"
```

Truncar strings largos — lo que tienes que hacer tú:

```
std::string formatear(std::string texto) {
    if ((int)texto.length() > 10)
        return texto.substr(0, 9) + "."; // 9 caracteres + "." = 10
    return texto;
}
```

```
std::cout << std::setw(10) << formatear(nombre) << "|";
```

Una fila completa:

```
std::cout << std::right;
std::cout << std::setw(10) << indice          << "|";
std::cout << std::setw(10) << formatear(fn)    << "|";
std::cout << std::setw(10) << formatear(ln)    << "|";
std::cout << std::setw(10) << formatear(nn)    << std::endl;
```

Resultado para un contacto con nombre "Alice":

```
0|      Alice|      Smith|      Ali|
```

PARTE 8 — ARGV Y ARGV

28. Cómo recibe el programa datos desde la terminal

Ya los conocías de C. En C++ funcionan exactamente igual:

```
int main(int argc, char **argv) {  
    // argc - número de argumentos (incluye el nombre del programa)  
    // argv - array de strings con los argumentos  
}
```

Cuando ejecutas `./megaphone "hola mundo" test`:

```
argc = 3  
argv[0] = "./megaphone"  
argv[1] = "hola mundo"  
argv[2] = "test"
```

Para convertir `argv[i]` (que es `char *`) a `std::string`:

```
std::string argumento = argv[1];    // conversión automática
```

El `ex00` (Megaphone) recorre todos los argumentos, los convierte a `std::string` y los imprime en mayúsculas:

```
for (int i = 1; i < argc; i++) {  
    std::string arg = argv[i];  
    for (int j = 0; j < (int)arg.length(); j++)  
        std::cout << (char)toupper(arg[j]);  
}  
std::cout << std::endl;
```

Si no hay argumentos (`argc == 1`), imprime el mensaje especial:

```
if (argc == 1) {
    std::cout << "* LOUD AND UNBEARABLE FEEDBACK NOISE *" << std::endl;
    return 0;
}
```

PARTE 9 — LOS EJERCICIOS EXPLICADOS UNO A UNO

29. Ex00: Megaphone

Lo que pide: recibir strings como argumentos y imprimirlos en mayúsculas. Sin argumentos, imprimir el mensaje de ruido.

Lo nuevo que usa: `argc / argv`, `std::string`, `toupper`, `std::cout`.

El flujo:

```
./megaphone "hola" "mundo"
argc = 3
i=1: "hola" → toupper cada carácter → "HOLA"
i=2: "mundo" → toupper cada carácter → "MUNDO"
cout: HOLAMUNDO
```

Espera — ¿“HOLAMUNDO” sin espacio entre argumentos? Sí. Fíjate en la salida esperada del subject:

```
$> ./megaphone "shhhhh... I think the students are asleep..."
SHHHHH... I THINK THE STUDENTS ARE ASLEEP...

$> ./megaphone Damnit " ! " "Sorry students, I thought this thing was off."
DAMNIT ! SORRY STUDENTS, I THOUGHT THIS THING WAS OFF.
```

En el segundo ejemplo hay un espacio al principio del resultado (“DAMNIT !”). Ese espacio viene del argumento “ ! ” — el espacio es parte del argumento, no lo añades tú.

30. Ex01: PhoneBook — la clase Contact

Lo que pide: una agenda que guarda hasta 8 contactos con nombre, apellido, apodo, teléfono y secreto oscuro.

La clase `Contact` guarda los datos de un contacto:

```
class Contact {
    private:
        std::string _firstName;
        std::string _lastName;
        std::string _nickname;
        std::string _phoneNumber;
        std::string _darkestSecret;

    public:
        Contact();
        ~Contact();

        // setters - para guardar datos
        void setFirstName(std::string name);
        void setLastName(std::string name);
        void setNickname(std::string name);
        void setPhoneNumber(std::string phone);
        void setDarkestSecret(std::string secret);

        // getters - para leer datos
        std::string getFirstName(void) const;
        std::string getLastName(void) const;
        std::string getNickname(void) const;
        std::string getPhoneNumber(void) const;
        std::string getDarkestSecret(void) const;

        // utilidades
        bool isEmpty(void) const;    // ¿este contacto tiene datos?
};
```

El método `isEmpty()` lo usará `PhoneBook` para saber si una posición del array está vacía o tiene un contacto real.

31. Ex01: PhoneBook — la clase PhoneBook

La clase `PhoneBook` gestiona el array de contactos:

```
class PhoneBook {
```

```

private:
    Contact _contacts[8];    // array estático — sin new, sin delete
    int     _nextIndex;      // próxima posición donde añadir

public:
    PhoneBook();
    ~PhoneBook();

    void addContact(void);
    void searchContact(void) const;
};

```

El array estático — por qué no usar new:

El subject dice explícitamente: “dynamic allocation is forbidden.” El array `_contacts[8]` vive en el stack — se crea cuando creas el `PhoneBook` y se destruye cuando este se destruye. Sin `new`, sin `delete`, sin memory leaks.

Para que `Contact _contacts[8]` funcione, `Contact` debe tener un constructor por defecto (sin parámetros) — el compilador lo llama para construir cada uno de los 8 elementos al crear el array.

La lógica de reemplazar el más antiguo:

```

void PhoneBook::addContact(void) {
    // pedir los datos al usuario
    // ...

    // guardar en la posición _nextIndex
    _contacts[_nextIndex].setFirstName(nombre);
    // ...

    // avanzar al siguiente índice de forma cíclica
    _nextIndex = (_nextIndex + 1) % 8;
    //                               ^^^
    // % 8 hace que cuando llega a 8, vuelva a 0
    // 0→1→2→3→4→5→6→7→0→1→2... (ciclo)
}

```

El operador `%` (módulo) devuelve el resto de la división. $8 \% 8 = 0$, $9 \% 8 = 1$. Esto hace que `_nextIndex` cicle de 0 a 7 y vuelva a 0 — sobrescribiendo el más antiguo.

El comando SEARCH:

Muestra la lista de contactos en columnas de 10 caracteres, pide un índice, y muestra ese contacto completo:

```
0|      Alice|      Smith|      Ali|
1|      Bob|      Jones|      Bobs|
Elige índice: 0
First name: Alice
Last name: Smith
...
```

32. Ex02: Account — static en acción

Este ejercicio es diferente — te dan el `.hpp` y el log de salida esperado, y tienes que escribir el `.cpp` que los hace coincidir. El ex02 no es obligatorio para pasar el módulo.

Lo que demuestra el ex02 es el uso de atributos y métodos estáticos. La clase `Account` representa una cuenta bancaria. Los datos totales (número de cuentas, dinero total) son estáticos — pertenecen a “todas las cuentas”, no a ninguna en particular:

```
class Account {
    private:
        static int _nbAccounts;    // total de cuentas — uno para todas
        static int _totalAmount;   // dinero total — uno para todas
        static int _totalNbDeposits;
        static int _totalNbWithdrawals;

        int _accountIndex;        // índice de esta cuenta — uno por cuenta
        int _amount;              // saldo de esta cuenta — uno por cuenta
        int _nbDeposits;
        int _nbWithdrawals;

    public:
        Account(int initial_deposit);
        ~Account(void);

        static void displayAccountsInfos(void); // método estático — no necesita
        void makeDeposit(int deposit);
        bool makeWithdrawal(int withdrawal);
        // ...
};
```

Los métodos estáticos:

Un método `static` se puede llamar sin crear ningún objeto:

```
Account::displayAccountsInfos();    // muestra el total de todas las cuentas
// llamada con Account:: — sin ningún objeto delante
```

Un método estático no tiene `this` — no sabe qué objeto es el "actual". Solo puede acceder a atributos estáticos, no a los de instancia.

GLOSARIO COMPLETO

Término	Definición
C++	Extensión de C que añade OOP — todo lo de C más clases, objetos y más
OOP	Object-Oriented Programming — agrupar datos y funciones en clases
Namespace	Etiqueta que agrupa nombres para evitar conflictos — <code>std::</code> es el de la librería estándar
<code>::</code>	Operador de resolución de ámbito — <code>std::cout</code> = "el cout dentro de std"
<code>std::cout</code>	Objeto que representa la pantalla — reemplaza a <code>printf</code>
<code>std::cin</code>	Objeto que representa el teclado — reemplaza a <code>scanf</code>
<code>std::endl</code>	Salto de línea + vaciar buffer — reemplaza a <code>'\n'</code> en cout
<code><<</code>	Operador de inserción — manda datos al stream (cout)
<code>>></code>	Operador de extracción — lee datos del stream (cin)
<code>std::string</code>	Clase de C++ para cadenas de texto — reemplaza a <code>char*</code>
<code>.length()</code>	Número de caracteres del string
<code>.empty()</code>	True si el string está vacío
<code>.find(s)</code>	Posición donde empieza <code>s</code> en el string — <code>npos</code> si no encontrado
<code>std::string::npos</code>	Constante especial que significa "no encontrado"
<code>.substr(pos, n)</code>	Devuelve <code>n</code> caracteres desde posición <code>pos</code>

<code>.erase(pos, n)</code>	Borra n caracteres desde posición pos
<code>.insert(pos, s)</code>	Inserta el string s en posición pos
<code>toupper(c)</code>	Convierte un char a mayúscula — de <cctype>
<code>tolower(c)</code>	Convierte un char a minúscula — de <cctype>
Iterador	Cursor que señala a un elemento dentro de una colección
<code>begin()</code>	Iterador al primer elemento
<code>end()</code>	Iterador a la posición DESPUÉS del último — no al último
Clase	Molde para crear objetos — agrupa datos y funciones
Objeto / Instancia	Elemento concreto creado a partir de una clase
Atributo	Variable que pertenece a una clase
Método	Función que pertenece a una clase
<code>private</code>	Solo accesible desde dentro de la clase
<code>public</code>	Accesible desde cualquier sitio
Encapsulación	Proteger los datos con private — acceso solo mediante métodos
Constructor	Método especial que se llama automáticamente al crear un objeto
Destructor	Método especial que se llama automáticamente al destruir un objeto
Lista de inicialización	<code>: atributo(valor)</code> antes del cuerpo — inicializa atributos eficientemente
<code>this</code>	Puntero al objeto actual dentro de un método
Getter	Método que devuelve el valor de un atributo privado
Setter	Método que modifica el valor de un atributo privado
<code>const (método)</code>	El método no modifica ningún atributo — <code>void f() const;</code>
<code>static (atributo)</code>	Existe una sola vez para toda la clase — compartido por todos los objetos
<code>static (método)</code>	Se puede llamar sin ningún objeto — no tiene <code>this</code>

<code>.hpp</code>	Header — declara qué existe en la clase
<code>.cpp</code>	Implementación — define cómo funciona cada cosa
Include guard	Evita que un header se incluya dos veces
<code>#pragma once</code>	Include guard moderno — una sola línea
<code>std::getline</code>	Lee una línea entera incluyendo espacios
<code>cin.ignore()</code>	Descarta el '\n' que queda en el buffer tras cin >>
<code>#include <iomanip></code>	Header para formatear la salida (setw, right, left)
<code>std::setw(n)</code>	Establece el ancho del siguiente campo a n caracteres
<code>std::right</code>	Alinea los campos a la derecha (persiste)
Array estático	Array de tamaño fijo en el stack — sin new ni delete
% (módulo)	Resto de la división — útil para hacer índices cíclicos
LIFO	Last In First Out — los objetos del stack se destruyen al revés de como se crean
UpperCamelCase	Estilo para nombres de clase — <code>PhoneBook</code> , <code>Contact</code>
lowerCamelCase	Estilo para métodos — <code>setNombre</code> , <code>getApellido</code>